Data Access Guide

Analytical databases `wansoft` and `zenput` | Fonda Argentina | Version 1.4 (2026-10-16)

Contents

1. What is where 2
2. Golden rules (read before writing any query) 3
3. Branch crosswalk 4
4. Which table answers which question 6
   - 4.1 Available history 7
5. Domains 9
   - 5.1 Sales (Wansoft, all branches) 9
   - 5.2 Costs (Wansoft or Odoo, by branch) 9
   - 5.3 Purchases (Wansoft history + Odoo, merged) 11
   - 5.4 Inventory 13
   - 5.5 Zenput (zenput database) 14
   - 5.6 Governance tables 15
6. Do not use 15
7. Freshness and schedule 15
8. Access 16
9. Still missing (under construction) 16
10. Version history 16

Appendix A. Table schemas 18
   - A.1 Sales 18
   - A.2 Costs 22
   - A.3 Purchases and invoices 25
   - A.4 Inventory 31
   - A.5 Governance and dimensions 38
   - A.6 Zenput (database zenput) 43

Audience: external teams building their own ETL on top of these databases (for example, a question-and-answer / chatbot layer). It tells you **which table to use for each business question, which fields matter, how tables join, and the traps to avoid**. Every rule here was checked against the live schema and data on 2026-09-28; the guide is current as of **2026-10-16**. Every table and column it names was confirmed to exist in production on 2026-10-07, and the full schema of every table (Appendix A) was taken from production on 2026-10-13. PDF version: data-access-guide.en.pdf.

Version 1.4 (2026-10-16). Every change to this guide raises its version number and is recorded in Section 10.

What is new in version 1.4 (compared with 1.3 of 2026-10-13):

- **Unified inventory movements:** new table `analytics_inventory_movements`, one row per movement (purchase, sale, waste, consumption, production, adjustment, transfer) of Wansoft and Odoo, with full history, a normalized type, cost and a common product code (5.4).
- **Wansoft code of an Odoo product:** new table `analytics_odoo_product_code`, refreshed every night (5.4). Corrects version 1.3, which pointed at `inventory_mapping_dictionary`, a table that is not granted.
- **Section 6 corrected:** `odoo_inventory_quant_daily` is to be used; the list of tables not to use no longer includes it.
- **Access:** the 4 tables that were missing are now granted (`costs_source_by_company`, `odoo_inventory_quant_daily` and the two `wansoft_parallel_*`) plus the 2 new ones; 28 `wansoft` tables in total, all in Appendix A.
- **Section 9 updated:** the unified movements are live; next steps: access limited to a fixed IP and an encrypted connection (TLS).

1. What is where

Two MySQL (MariaDB 10.4) databases on the same server:

Database	Content
`wansoft`	Sales, costs, purchases, inventory and the unified analytical layer
`zenput`	Operational checklists (forms, answers) and tasks from Zenput

Source systems behind them:

Source	What it provides	How it arrives
Wansoft (POS/ERP)	Sales and cash closing for all branches; costs, purchases and inventory for branches not yet on Odoo, and the others' history	Daily download via its SOAP API
Odoo	Purchases, inventory and costs of the branches operating on Odoo, from their start date	Daily read-only extraction
Zenput	Checklists and tasks	Daily API download

Layers inside `wansoft`:

Layer	Tables	Use it for

Operational tables	`get*`, `costeomensual*`	Sales, cash closing and Wansoft-side detail (Wansoft copies); costs of every branch, computed from Wansoft or Odoo per branch (Section 5.2)
Canonical	`canonical_purchase_*`	Internal: Odoo and Wansoft purchases merged into one shape. Prefer the analytics layer
Analytics (business-ready)	`analytics_*`	Purchases and inventory for all branches, both sources merged
Dimensions	`dim_company_analytical`, `dim_product`, `dim_vendor`, `dim_time`	Descriptions and attributes for the analytics layer

2. Golden rules (read before writing any query)

1. **A branch is identified three different ways** depending on the table. Use the crosswalk in Section 3; never join tables on branch names blindly.
2. Short key (`Acoxpa`, `Oceanía`): sales tables (`Sucursal`) and the analytics layer (`company_source_key`).
3. **Numeric Wansoft id** (`5320`): costs (`subsidiary_id`) and the inventory tables — where the id is stored in a column confusingly named `subsidiary_name`.
4. Long Wansoft name (`Fonda Argentina - Acoxpa`): costs (`subsidiary_name`) and invoices (`getexpenses_factura.Sucursal`).
5. **`getexpenses_factura.Sucursal` contains the same branch spelled two ways** because of historical encoding damage (`Cancún` and `Canc??n`, `Oceanía` and `Ocean??a`). Map it to the crosswalk before grouping.
6. **Sales tables store amounts and dates as text** (`varchar`). Cast them: `CAST(Total AS DECIMAL(14,2))`, `DATE(Fecha)` (format `2026-09-01T00:00:00`).
7. **Analytics tables carry `include_in_business_views`.** Filter `include_in_business_views = 1` for business numbers; the other rows are kept for audit (internal providers, unmapped products, review cases) and carry an `exclude_reason`.
8. **Purchases and inventory come from two systems; the analytics layer already merges them without overlap** (Section 5.3). Do not rebuild the merge from raw tables.
9. **Data is loaded once a day, at about 01:30, through the previous day.** Each run also re-checks recent days (sales 10 days, costs and butchery 10, Wansoft invoices/inventory/cash closing 5, purchases 35), so the last few days can still change.
10. Costs tables are **cumulative snapshots**, not daily amounts (Section 5.2).

3. Branch crosswalk

Wansoft id	Short key (`sucursal`, `company_source_key`)	Wansoft long name (costs)	Odoo company	Purchases & inventory source	Odoo start date
5320	Acoxpa	Fonda Argentina - Acoxpa	FONDA COSTA NERA	Wansoft → **Odoo from 2026-10-01**	2026-10-01
4959	Aeropuerto	Fonda Argentina - Aeropuerto	FONDA ARGENTINA AEROPUERTO	Wansoft	—
4960	Antenas	Fonda Argentina - Antenas	FONDA ARGENTINA LAS ANTENAS	Wansoft → **Odoo from 2026-10-01**	2026-10-01
6175	Cancun	Fonda Argentina - Cancún	—	Wansoft	—
12802	CentroMyJ	Fonda Argentina - Centro Mario y July	MARIO Y JULY	Odoo (born on Odoo)	2026-06-01
4958	Isabel La Católica	Fonda Argentina - Isabel La Católica	FONDA ARGENTINA	Wansoft → **Odoo from 2026-10-01**	2026-10-01
12057	La Esquina Coyoacán	Fonda Argentina - Coyoacan	FONDA ARGENTINA COYOACAN	Wansoft → **Odoo from 2026-10-01**	2026-10-01
4752	Metepec	Fonda Argentina - Tollocan	FONDA ARGENTINA TOLLOCAN	Wansoft	—
4433	Napoles	Fonda Argentina - Nápoles	FONDA ARGENTINA POLYFORUM	Wansoft	—
5943	Oceanía	Fonda Argentina - Oceanía	FONDA ARGENTINA ENCUENTRO OCEANIA	Wansoft → **Odoo from 2026-10-01**	2026-10-01
6174	Playa del Carmen	Fonda Argentina - Playa del Carmen	—	Wansoft	—
12806	Puebla	Fonda Argentina - Puebla	FONDA ARGENTINA PUEBLA	Odoo (born on Odoo)	2026-06-10
5319	San Jeronimo	Fonda Argentina – San Jerónimo	FONDA ARGENTINA SAN JERONIMO	Wansoft → **Odoo from 2026-10-01**	2026-10-01
5321	Taquería parroquia	Fonda Argentina – Taquería Parroquía	—	Wansoft	—
4962	Taquería Viaducto	Fonda Argentina - Taqueria Viaducto	—	Wansoft	—
6560	Tepeyac	Fonda Argentina - Tepeyac	FONDA ARGENTINA MAQ	Wansoft → **Odoo from 2026-10-01**	2026-10-01
5396	Versalles	Fonda Argentina - Taquería Exhibimex	—	Wansoft	—
5318	Vía Vallejo	Fonda Argentina – Vía Vallejo	FONDA ARGENTINA VALLEJO	Wansoft → **Odoo from 2026-10-01**	2026-10-01
4961	Viaducto	Fonda Argentina - Viaducto	FONDA ARGENTINA VIADUCTO	Wansoft	—

Notes:

- Names differ between systems for the same branch: Metepec = "Tollocan", Napoles = "Polyforum", Versalles = "Taquería Exhibimex", Acoxpa = "Costa Nera" in Odoo. Long names use either a hyphen (`-`) or an en dash (`–`).
- `7697` "Taqueria San Fernando" appears in historical costs only; it is not one of the 19 current branches.
- **Sales always come from Wansoft for every branch**, whatever the purchases source.
- **New branches** (today Puebla and CentroMyJ, and every branch opened from now on) are born on Odoo: purchases, inventory and costs come from Odoo; in Wansoft they only record sales and daily closings.
- **Costs:** every branch operating on Odoo, migrated or new, takes its cost from Odoo **from its start date on**, and from Wansoft before that date. **Temporary exception: Antenas** keeps Wansoft costs while its Odoo database is repaired (it was the prototype). **Isabel La Católica, San Jeronimo and Vía Vallejo** keep Wansoft costs until Odoo has their cost of sales (they wait for their opening inventory balances); each one switches to Odoo by itself on the first day Odoo has data. See Section 5.2.
- Internal providers **El Bodegón de Fito** and **Las Empanadas de María Eva** (central kitchens) are excluded as buying companies from business views.
- The same information, as data: `dim_company_analytical` and `analytics_company_domain_coverage` (Section 5.6). They use the short key and do not carry the Wansoft id; use this table for that link.

4. Which table answers which question

Question	Table	Filter / note
Sales, tickets, covers per day/branch	`getallordenesbyday_new_venta`	one row per ticket
What was sold (dishes, groups)	`getallordenesbyday_new_detalleventa`	one row per line
Payment methods, tips	`getallordenesbyday_new_pago`	one row per payment
Official daily close (Z report), cancellations, courtesies, discounts	`getglobalcashclosing`	one row per branch per closing
Monthly cost of sales, waste, margin	`costeomensual`	month-to-date snapshot; take the last day
Weekly cost	`costeomensual_semanapyq`	week-to-date snapshot
Daily cost of sales	`gettotalcostbydate`	one row per branch per day
Butchery yields	`gettablajeriareport`	history of Wansoft branches only; being phased out (see 5.2)
Purchases (goods) for any branch, any period	`analytics_purchase_order_lines` / `analytics_purchase_orders` / `analytics_purchase_daily_company_product`	`include_in_business_views = 1`
Spend by accounting account (Costo operativo, Gastos directos, ...)	`getexpenses_factura`	Wansoft only (see 5.3)
Inventory entries/exits detail (Wansoft)	`getinputinventory_entrada`, `getoutgoinginventory_salida`	Wansoft branches
What the Odoo branches keep capturing in Wansoft (comparison only)	`wansoft_parallel_inputinventory_entrada`, `wansoft_parallel_outgoinginventory_salida`	from 2026-10-01; never in totals (see 5.4)
Current stock, all 19 branches	`analytics_inventory_balance` (and `analytics_inventory_current_product_location` for Odoo by location)	`include_in_business_views = 1`; today's snapshot only
Stock on a past date, Wansoft branches	`getinputinventory_entrada` – `getoutgoinginventory_salida` up to that date	see 5.4
Stock on a past date, Odoo branches	`odoo_inventory_quant_daily`	from 2026-10-08 (see 5.4)
Checklist results	`zenput.submissions` + `zenput.submission_answers`	—
Tasks	`zenput.zenput_tasks`	—
Which source feeds each branch	`analytics_company_domain_coverage`	—
From which day each branch's cost comes from Odoo	`costs_source_by_company`	`odoo_cost_start_date` NULL = Wansoft
Every inventory movement (purchases, sales, waste, consumption, production, adjustments, transfers), Wansoft + Odoo, full history	`analytics_inventory_movements`	`include_in_business_views = 1`; always filter by branch and dates (5.4)
Wansoft code of an Odoo product	`analytics_odoo_product_code`	one row per Odoo product (5.4)

4.1 Available history

Start dates measured in production on 2026-09-28; the go-live of October 1 did not change them (it only removed duplicate copies of rows). Tables keep filling daily through the previous day.

Data	Table	Source	Since	Notes
Sales: tickets, lines, modifiers	`getallordenesbyday_new_venta`, `_new_detalleventa`, `_new_modificador`	Wansoft	2021-09-01	2021-12-14 to 2021-12-31 has almost no tickets (a gap in the source history, accepted)
Sales: payments per ticket	`getallordenesbyday_new_pago`	Wansoft	2025-01-01	No payments per ticket before 2025; use the cash closing
Daily cash closing	`getglobalcashclosing`	Wansoft	2022-01-01	—
Daily cost of sales	`gettotalcostbydate`	Wansoft or Odoo (5.2)	2021-07-31	—
Month-to-date cost	`costeomensual`	Wansoft or Odoo (5.2)	2022-01-01	Puebla and CentroMyJ have costs from July 2026 (5.2)
Week-to-date cost	`costeomensual_semanapyq`	Wansoft or Odoo (5.2)	2024-01-02	—
Butchery yields	`gettablajeriareport`	Wansoft	2022-01-01	Being phased out (Section 5.2)
Supplier invoices by accounting account	`getexpenses_factura`	Wansoft	2019-02-18	The oldest history; only while a branch enters purchases in Wansoft
Inventory entries	`getinputinventory_entrada`	Wansoft	2021-09-01	—
Inventory exits	`getoutgoinginventory_salida`	Wansoft	2020-11-30	About 1.1 million rows have an empty `Fecha` (`0000-00-00`): adjustments and waste whose business date is in `FechaReal`. **Do not exclude them**: use `COALESCE(NULLIF(Fecha, '0000-00-00'), FechaReal)` (5.4)
Unified purchases	`analytics_purchase_order_lines`, `analytics_purchase_orders`, `analytics_purchase_daily_company_product`	**Wansoft + Odoo**	**2021-09-01**	Wansoft until each branch's Odoo start date, Odoo from then on (2026-10-01 for the migrated branches; Puebla and CentroMyJ since they opened in June). Checked year by year: same rows and amounts as production's Wansoft invoices
Unified stock	`analytics_inventory_current_product_location`, `analytics_inventory_balance`	**Wansoft + Odoo**	Current snapshot	No history by design; the Wansoft balance is recomputed nightly from every movement since 2020-11-30
Parallel Wansoft inventory (8 migrated branches)	`wansoft_parallel_inputinventory_entrada`, `wansoft_parallel_outgoinginventory_salida`	Wansoft	2026-10-01	Comparison with Odoo only; rows before 2026-10-01 are September movements, read them from the main tables (5.4)
Odoo stock day by day	`odoo_inventory_quant_daily`	Odoo	2026-10-08	One raw capture per night (product × location); no Odoo stock history before that date

Checklists	`zenput.submissions`, `zenput.submission_answers`	Zenput	2025-06-11	—
Tasks	`zenput.zenput_tasks`	Zenput	2025-06-03	—
Unified inventory movements	`analytics_inventory_movements`	**Wansoft + Odoo**	2020-09-29	Wansoft until each branch's Odoo start and Odoo from then on (Puebla from 2026-06-09); what was captured in parallel is kept with `include_in_business_views = 0`
Calendar	`dim_time`	—	2020-01-01	Until 2035-12-31

5. Domains

5.1 Sales (Wansoft, all branches)

`getallordenesbyday_new_venta` — ticket header.

Field	Meaning
`Sucursal`	Branch short key
`Movimento`	Ticket id within the branch (sic, no "i"); join key
`Fecha`	Business date, text `YYYY-MM-DDT00:00:00`
`Total`, `Subtotal`, `IVA`, `IEPS`, `Descuento`	Amounts, text
`Personas`	Covers
`Mesa`, `Mesero`, `Orden`, `Terminal`	Table, waiter, order number, terminal
`TipoOrden`	`Restaurant`, `Para llevar`, `eCommerce`
`HoraApertura`, `HoraCierre`	Open / close time
`Estatus`	Always `0` in current data

Joins (both columns needed, the ticket id repeats across branches): `venta.Sucursal = detalle.Sucursal AND venta.Movimento = detalle.Movimiento_Id`; same for `getallordenesbyday_new_pago` and `getallordenesbyday_new_modificador`.

`getallordenesbyday_new_detalleventa` — lines: `Platillo`, `CodigoPlatillo`, `Grupo`, `TipoGrupo` (e.g. food / drinks), `Cantidad`, `PrecioUnitario`, `Total`, `Costo`, `Cortesia`, `Hora`.

`getallordenesbyday_new_pago` — `MetodoDePago`, `Total`, `Propina`, `Terminal`.

Waiters: delivery and app orders are attributed to a **placeholder waiter** whose name varies by branch (`APLICACIONES`, `Apps Llevar`). Exclude it when ranking real waiters.

```
SELECT Sucursal, DATE(Fecha) AS dia,
      COUNT(*) AS tickets,
      SUM(CAST(Personas AS UNSIGNED)) AS comensales,
      SUM(CAST(Total AS DECIMAL(14,2))) AS venta
FROM getallordenesbyday_new_venta
WHERE Fecha >= '2026-09-01' AND Fecha < '2026-10-01'
GROUP BY Sucursal, DATE(Fecha);
```

`getglobalcashclosing` — the official close per branch (`subsidiary_id`, `fecha_corte`): `total_ventas`, `no_ordenes`, `total_personas`, averages, and counts/amounts of courtesies, cancellations, discounts, voids, promotions. Amounts here are **sale value**. Use it to reconcile daily totals.

5.2 Costs (Wansoft or Odoo, by branch)

Where each branch's cost comes from:

Branch type	Cost source today
Wansoft only	Wansoft cost report

Migrated to Odoo (Acoxpa, Tepeyac, Oceanía, La Esquina Coyoacán)	Wansoft through 2026-09-30; Odoo from 2026-10-01
October wave (Isabel La Católica, San Jeronimo, Vía Vallejo)	Wansoft until Odoo has their cost of sales; from that day (different for each), Odoo. Until then their October Wansoft rows carry merma and consumo as 0, since these are no longer captured there
New, born on Odoo (Puebla, CentroMyJ and any branch opened later)	Odoo since it opened. Odoo has no cost of sales for CentroMyJ before July 2026 nor for Puebla before 2026-07-27: those days are 0
Antenas (temporary exception)	Wansoft, while its Odoo database is repaired

Odoo cost is computed from Odoo accounting (direct-cost accounts) and stored in the same tables. It fills `CostoTotal`, `CostoDeProductosVendidos` and `CostoDeMerma`; `CostoDeCortesías` and `CostoDeCancelaciones` come from the cash closing; the other columns stay empty (no Odoo equivalent). **In Odoo rows the columns with no equivalent are NULL**: use `COALESCE(column, 0)` when adding or subtracting, or the result is NULL (e.g. `CostoTotal - CostoDeConsumo` is NULL for Puebla). The switch is per day: days before the start date keep Wansoft's cost. When the switch falls in the middle of a month or week, the accumulated tables (`costeomensual`, `costeomensual_semanapyq`) stay whole: they add Wansoft's part up to the day before the switch to Odoo's from that day, with no day counted twice. On Odoo rows `CostoTotal` no longer includes consumo.

Which source each branch uses, as data: the table `costs_source_by_company` (rewritten every night with the pipeline's own rule) has, per branch, `company_source_key`, `wansoft_subsidiary_id`, `odoo_company_id`, `odoo_cost_start_date` (first day on Odoo cost; NULL = always Wansoft) and `reason` (`wansoft`, `policy`, `exception`, `auto_switch_pending`, `switched`, `no_policy`). Use it instead of copying the rules of this section.

Costs of recent days are not complete yet, and each source shows it differently:

- **Wansoft:** its cost grows while it processes the inventory deduction of what was sold. `CostoIdealDeProductosPendientesDeRebaja` (in `costeomensual` and `costeomensual_semanapyq`) is the cost not yet deducted; when it is 0 the period is complete. All of September 2026 is already at 0; in early October 1% to 24% was pending depending on the branch, more on weekends.
- **Odoo:** its cost comes from each day's customer invoices (the Wansoft sales passed to Odoo), which are created with a lag: in September 2026, 40% to 70% of a day's sales was invoiced the next day, 70% to 100% after 7 days, and everything by the month-end closing. To know whether a day is complete, compare Odoo's invoiced amount with Wansoft's net sales for that day. The pipeline re-reads Odoo for the current month (and the previous one until the 10th), so after the closing the tables are complete.

If you need a cost for recent days, mark it as an estimate or wait until the period is complete.

Keyed by `subsidiary_id` (Wansoft id). All three are **snapshots**:

Table	One row =	How to read it
`costeomensual`	Month-to-date cumulative, from the 1st through `created_date`	Month total = the row with the latest `created_date` of that month
`costeomensual_semanapyq`	Week-to-date cumulative, Monday through the day **before** `created_date`	Week total = latest row of the week
`gettotalcostbydate`	Cost of sales of the single day `created_date`	Sum over days

`mes_ano` is text `MM-YYYY`. Unique per (`subsidiary_id`, `created_date`).

Fields in `costeomensual` and how the owner's reports use them:

Report figure	Formula

Costo Total	`CostoTotal - COALESCE(CostoDeConsumo, 0)`
Costo Teórico	`CostoDeProductosVendidos`
Gasto de Venta	`CostoDeConsumo`
Costo de Mermas	`CostoDeMerma`
Cortesías / Cancelaciones	`CostoDeCortesías`, `CostoDeCancelaciones` (sale value, from the cash closing)

```
-- Month cost per branch: last snapshot of the month
SELECT c.subsidiary_id, c.subsidiary_name, c.created_date,
      c.CostoTotal - COALESCE(c.CostoDeConsumo, 0) AS costo_total,
      c.CostoDeProductosVendidos AS costo_teorico
FROM costeomensual c
JOIN (SELECT subsidiary_id, MAX(created_date) AS d
     FROM costeomensual
     WHERE created_date >= '2026-08-01' AND created_date < '2026-09-01'
     GROUP BY subsidiary_id) last ON last.subsidiary_id = c.subsidiary_id AND last.d = c.created_date;
```

Butchery yields (`gettablajeriareport`) are being phased out. They only exist for branches that enter purchases and inventory in Wansoft: new branches (Puebla, CentroMyJ) have none, and migrated branches will stop having them once they stop entering purchases in Wansoft. Odoo does not record butchery this way: it applies calculated yields, and at month end the real inventory is checked against the calculated one. Use this table only as history for Wansoft branches.

5.3 Purchases (Wansoft history + Odoo, merged)

Use the analytics layer. It already merges both sources with this rule per branch:

- Branch purchasing in Wansoft: Wansoft invoices-type entries (`getinputinventory_entrada` with `TipoEntrada = 'Factura'`).
- Branch migrated to Odoo: Wansoft history **before** its Odoo start date (Section 3), then confirmed Odoo purchase orders (`state` purchase/done) from that date on. No overlap, no gap.
- Internal providers as buying company are excluded; a branch buying *from* an internal provider is kept.

`source_system` (`wansoft`/`odoo`) tells where each row came from; `final_purchase_source_status` explains why it was kept (for example `wansoft_history_before_odoo`).

When a purchase appears (important when reading recent days):

- **Odoo:** an order counts once it is **confirmed**; drafts and requests for quotation never count. It enters with the load of the night after it is confirmed, **under its order date**: an order dated day 5 and confirmed on day 7 appears in the table on day 8, but on day 5. So purchases of recent days can keep growing.
- **Wansoft:** a purchase counts when its **invoice is captured**, usually when the goods arrive.
- For the same goods the two moments can differ. The main case is **El Bodegón de Fito** (central kitchen), which invoices late: Odoo counts its orders when confirmed, Wansoft only when the invoice arrives.

Unclassified purchases (from the load of the night of 2026-10-07 to 10-08): a real purchase counts in the business views even if its vendor or product is not yet classified in the catalogs. `catalog_status` says which state it is in: `catalogado`, `producto_por_clasificar` (the product exists but its classification still has to be confirmed), `producto_sin_catalogo`, `proveedor_sin_catalogo` or `proveedor_y_producto_sin_catalogo`. Use it to show those purchases apart in a breakdown by category; the totals already include them. Only deliberately excluded products and

purchases made by the internal providers stay out (`include_in_business_views = 0`). The catalogs are rebuilt every night. Since October 8, a Wansoft purchase carrying its Wansoft code and department is `catalogado` even when its link to the equivalent Odoo product is still pending approval: `producto_por_clasificar` is left for products that really lack a classification.

Table	Grain	Key fields
`analytics_purchase_order_lines`	purchase line	`company_source_key`, `order_date`, `vendor_name`, `product_name`, `wansoft_code`, `wansoft_department`, `product_qty`, `price_unit`, `price_subtotal` (before tax), `price_total` (with tax), `source_system`
`analytics_purchase_orders`	purchase order / invoice	same header fields plus totals
`analytics_purchase_daily_company_product`	branch × day × product	`business_price_subtotal_total`, `business_product_qty_total` (already business-filtered)

Join to dimensions with `product_analytical_key` → `dim_product`, `vendor_analytical_key` → `dim_vendor`, `company_analytical_key` → `dim_company_analytical`, `order_date_key` → `dim_time.date_key`.

```
SELECT company_source_key, DATE_FORMAT(order_date, '%Y-%m') AS mes,
      source_system, SUM(price_subtotal) AS compras_sin_iva
FROM analytics_purchase_order_lines
WHERE include_in_business_views = 1
 AND order_date >= '2026-01-01'
GROUP BY company_source_key, mes, source_system;
```

Example: the migrated branches (Acoxpa, Antenas, Tepeyac, Oceanía, Coyoacán) come back with `source_system = 'wansoft'` through September 2026 and with `'odoo'` from October, with no month repeated or missing; Puebla and CentroMyJ, born on Odoo, come back with `'odoo'` since they opened in June 2026.

`getexpenses_factura` is a different thing: every Wansoft supplier invoice classified by accounting account (`Cuenta`: Costo operativo, Gastos de venta, Gastos directos, Sueldos y salarios, Fletes, ...), including services, software and payroll that are not goods. Odoo purchase orders only cover goods. So:

- For goods purchases across both systems, use the analytics layer.
- To compare with Wansoft's "Compras" reports, use only `Cuenta = 'Costo operativo'`.
- **Odoo and Wansoft do not match each other for the same branch and month.** During the parallel capture, Odoo purchases came out 7% to 19% above Wansoft's `Costo operativo` (September 2026: Acoxpa +7.2%, Coyoacán +11.1%, Oceanía +18.5%). It is a difference between the two source systems, not a loading error, and it is mostly explained by when each system counts a purchase (see "When a purchase appears"): in one week of Acoxpa reviewed, most orders matched their invoice exactly and almost all of the difference was El Bodegón de Fito orders not yet invoiced; the rest, small differences between the order amount and the invoice amount. The analytics layer does not double it because it never mixes both systems for the same branch and day; do not compare Odoo against Wansoft expecting them to match.
- **Non-goods spend** (rent, services, software, payroll, freight...) of branches already operating on Odoo is **not** extracted from Odoo today. In `getexpenses_factura` those branches only have invoices up to their Odoo start date. From Odoo accounting only the cost of sales is taken (Section 5.2).

Some Wansoft invoices have an empty `Cuenta`. Amounts and dates in this table are text. **`Estatus = 'Por pagar'` does not mean unpaid:** invoices are rarely marked paid in Wansoft (payment is recorded elsewhere), so do not use that field for accounts payable.

5.4 Inventory

- **Today's stock, all 19 branches:** `analytics_inventory_balance`, one row per branch × Wansoft product code; `source_system` says where it comes from: `wansoft` (the 9 Wansoft branches: entries – exits since the first movement) or `odoo` (the 10 Odoo branches: Odoo's stock). `analytics_inventory_current_product_location` gives the Odoo side by location (warehouse, kitchen). Filter `include_in_business_views = 1`. **It is today's snapshot: rewritten every night, no history kept.**
- **Wansoft movements:** `getinputinventory_entrada` (entries; `TipoEntrada`: `Factura`, `Transferencia`, `Entrada con canal`, `Producto procesado`, `Ajuste de inventario`) and `getoutgoinginventory_salida` (exits, tens of millions of rows: always filter by date and `subsidiary_name`). Branch = Wansoft id in `subsidiary_name`; product = `CodigoProducto`; quantity `Cantidad`, cost `CostoUnitario`. Wansoft branches only; the 8 branches migrated to Odoo end on 2026-09-30.
- **Parallel Wansoft inventory (since the night of 12 to 13 October 2026):** the 8 branches migrated to Odoo (Acoxpa, Antenas, Tepeyac, Oceanía, La Esquina Coyoacán, Isabel La Católica, San Jerónimo, Vía Vallejo) keep capturing entries and exits in Wansoft. That capture is loaded into `wansoft_parallel_inputinventory_entrada` and `wansoft_parallel_outgoinginventory_salida`, with the same columns as the main tables. **It is for comparing Wansoft with Odoo only: never add it to stock or totals**; for these branches the official source from 2026-10-01 is Odoo. Use only dates from 2026-10-01 on: earlier rows are September movements, read them from the main tables.
- **Stock on a past date, Wansoft branches:** entries – exits up to that date, with the same rules as today's balance (no transfers, no purchase orders, no capture errors or rejected invoices). Exits with an empty `Fecha` are adjustments and waste dated in `FechaReal`: do not exclude them.

```
-- Stock of Coca Cola Mini (1000-105-203-005) at Viaducto (4961) at the close of 2026-09-30
SELECT
 (SELECT COALESCE(SUM(Cantidad), 0) FROM getinputinventory_entrada
   WHERE subsidiary_name = '4961' AND CodigoProducto = '1000-105-203-005'
     AND TipoEntrada NOT IN ('Orden de compra a proveedor', 'Transferencia')
     AND DATE(FechaEntrada) <= '2026-09-30')
 -
 (SELECT COALESCE(SUM(Cantidad), 0) FROM getoutgoinginventory_salida
   WHERE subsidiary_name = '4961' AND CodigoProducto = '1000-105-203-005'
     AND TipoSalida NOT IN ('Error de captura', 'Factura de egresos rechazada', 'Transferencia')
     AND COALESCE(NULLIF(Fecha, '0000-00-00'), FechaReal) <= '2026-09-30') AS stock;
-- 2,939. With '2026-10-07' it gives 3,170, equal to analytics_inventory_balance that night.
```

- **Odoo stock day by day:** `odoo_inventory_quant_daily`, from stock date 2026-10-08. One row per `stock_date`, location and Odoo product (`odoo_product_id`, `product_name`, `location_name`, `location_usage` `internal` or `transit`, `quantity`, `company_source_key`), without mappings. To read it in Wansoft codes, join `odoo_product_id` to `analytics_odoo_product_code` (below). **There is no Odoo stock history before 2026-10-08.**
- **Unified inventory movements (since version 1.4):** `analytics_inventory_movements`, one row per movement of one product in one branch, Wansoft and Odoo, with full history (about 40 million rows). Use it for flows: how much of each product was bought, sold, wasted or consumed, per branch and day, whatever the system.

Field	Meaning
`movement_date`	Business date of the movement
`company_source_key`	Branch (short key)
`source_system`	`wansoft` or `odoo`

`direction`	`in` (into the branch) or `out`
`movement_type`	Normalized type: `purchase_receipt`, `sale_consumption`, `sale_cancellation`, `waste`, `theft`, `consumption`, `production_in`, `production_out`, `transfer_in`, `transfer_out`, `adjustment`, `initial_inventory`, `other`
`source_movement_type`	The original type (Wansoft `TipoEntrada`/`TipoSalida`; Odoo source → destination)
`product_code`, `product_code_status`	Common product code: the Wansoft code (`wansoft`) or a temporary `ODOO-<id>` (`temporal_odoo`) while the Odoo product has no equivalence; once it does, the code changes across its whole history
`wansoft_code`, `odoo_product_id`, `product_name`, `unit_name`	Product and unit
`quantity`	Signed quantity (+ in, – out), in the product's unit
`unit_cost`, `total_cost`	Cost (Wansoft: unit cost; Odoo: valuation); `total_cost` signed
`counts_in_stock`	Whether the movement changes stock (Wansoft internal transfers and purchase orders do not)
`final_source_status`, `include_in_business_views`	As in purchases: filter `include_in_business_views = 1` for business figures. Migrated branches take Wansoft before their Odoo start and Odoo from then on; what was captured in parallel in the other system has `include_in_business_views = 0` (`wansoft_parallel_after_odoo`, `odoo_parallel_before_start`)

```
-- Waste and consumption per month of one branch, both systems
SELECT DATE_FORMAT(movement_date, '%Y-%m') AS month, movement_type,
      SUM(-quantity) AS quantity, SUM(-total_cost) AS cost
FROM analytics_inventory_movements
WHERE company_source_key = 'Acoxpa' AND include_in_business_views = 1
 AND movement_type IN ('waste', 'consumption')
 AND movement_date >= '2026-01-01'
GROUP BY month, movement_type;
```

Every night the Wansoft movements captured in the last 10 days and the whole Odoo side are reloaded, so recent days can still change. For stock use `analytics_inventory_balance` (today) and `odoo_inventory_quant_daily` (Odoo per day); unified daily stock is under construction (Section 9). - **Wansoft code of an Odoo product:** `analytics_odoo_product_code` (`odoo_product_id`, `odoo_product_name`, `wansoft_code`, `wansoft_product_name`), one row per Odoo product with a current, reviewed equivalence; rewritten every night, the same one `analytics_inventory_movements` uses. Use it to read `odoo_inventory_quant_daily` in Wansoft codes. **Do not use `dim_product` for this**: it includes equivalences still under review. - Detailed coverage per branch: `docs/inventory-coverage-by-branch.md` in the repository.

5.5 Zenput (`zenput` database)

Table	Grain	Key fields
`form_templates`	checklist template	`form_id`, `title`, `category_name`
`submissions`	one filled checklist	`submission_id`, `form_template_id` → `form_templates.form_id`, `location_name`, `user_display_name`, `date_submitted`
`submission_answers`	one answer	`submission_id`, `title` (question), `field_type`, `value_as_string`
`zenput_tasks`	task	`task_id`, `title`, `account_name` (location), `status_name`, `assignee_display_name`, `date_due`, `is_closed`, `is_completed_late`

`location_name` / `account_name` are Zenput's own location names, not the short key; some Zenput locations have no POS branch.

5.6 Governance tables

- `dim_company_analytical`: one row per company, flags (`is_active_branch`, `is_internal_provider`, `is_final_operating_branch`), source per domain, `operational_start_date`.
- `analytics_company_domain_coverage`: which source feeds each domain per branch, with counts and a `coverage_status`.
- `costs_source_by_company`: from which day each branch's cost comes from Odoo (`odoo_cost_start_date`, NULL = Wansoft) and why (`reason`); rewritten every night (Section 5.2).
- `dim_time`: calendar (`date_key` = `YYYYMMDD`), weeks, months, ISO weeks.

6. Do not use

Tables	Why
`canonical_purchase_*`, `odoo_purchase_*`, `odoo_stock_move_snapshot` and the `odoo_inventory_*` **except `odoo_inventory_quant_daily`**	Intermediate layers; the analytics tables are the business-ready result
`inventory_*`, `product_catalog_mapping`, `backup_product_catalog_mapping`, `product_replacement_candidates`, `stg_*`	Mapping and review workbench; for the product equivalence use `analytics_odoo_product_code`
`odoo_company_migration_policy`, `odoo_cutover_validation_log`	Pipeline control
`getinventorybydepartment`, `getstockinventory_inventario`, `getinventorybyday_*`	Old snapshots, not refreshed daily
`vw_inventory_*`	Diagnostic views
`tmp_*`	Temporary

7. Freshness and schedule

- Daily pipeline at **01:30** (Mexico City time), about 40 minutes; data through the previous day. In production since 2026-10-01.
- Rolling re-checks: sales 10 days (reconciled against all of Wansoft's cash closings of the day; the latest day is validated the next night), costs and butchery 10 days (Odoo cost: the current month and, until the 10th, the previous one), Wansoft invoices/inventory/cash closing 5 days, Wansoft purchases 35 days. Odoo purchases are fully reloaded every night; an order appears the night after it is confirmed (section 5.3).
- Costs in Wansoft can be recalculated by Wansoft after the fact; the latest snapshot wins.
- Some branches (Antenas, Vía Vallejo, Coyoacán, Metepec, Taquería Parroquia, Versalles, Tepeyac) upload their cash closing to Wansoft only the next morning, even when they close at night: their `getglobalcashclosing` row arrives **one day late** (sales are on time in the tickets). Until it arrives, that day has no cancellations, courtesies or discounts; take sales from the tickets.
- Weekly backup Thursdays 18:00. Avoid heavy queries at 01:30–03:00.

8. Access

Access is through a dedicated **read-only** database user limited to the tables in Sections 4–5, provided separately (credentials are never written in this document). Query `getoutgoinginventory_salida` and the sales detail tables with date filters: they hold tens of millions of rows. Each user is limited to 4 simultaneous connections and 30 minutes per query.

The tables the user can read are exactly those of Appendix A, with all their columns. When a new table is added (Section 9), it is granted to the user and this guide and the appendix are updated.

9. Still missing (under construction)

State on 2026-10-16. The unified inventory movements are live (5.4). The tables below do not exist yet; when they do, they will be granted to the user and documented here.

What	Table	State	What it solves
Unified daily stock	`analytics_inventory_stock_daily`	Designed	Stock per branch × product × day since the first movement, both sources; replaces the entries – exits formula of Section 5.4 and covers the Odoo gaps before 2026-10-08
Unified supplier invoices	`analytics_expense_invoices`	Designed	Wansoft invoices by account + Odoo vendor bills; today the non-goods expenses of the Odoo branches are not extracted (5.3)

Other upcoming changes:

- **Aeropuerto and Taquería Viaducto move to Odoo on 2026-11-01** (as branches of the company FONDA ARGENTINA AEROPUERTO). From that day their purchases, inventory and costs come from Odoo, with the same rule as Section 3; sales stay in Wansoft.
- **Isabel La Católica, San Jerónimo and Vía Vallejo** keep Wansoft costs until Odoo has their cost of sales; `costs_source_by_company` shows the switch day (5.2).
- **Antenas** keeps Wansoft costs while its Odoo database is repaired.
- **Access security:** the user's access will be limited to a fixed IP and the connection will be encrypted (TLS); we will announce the date and what is needed to connect.

10. Version history

The most recent version comes first.

Version 1.4 (2026-10-16)

- **Unified inventory movements:** new table `analytics_inventory_movements`, one row per movement (purchase, sale, waste, consumption, production, adjustment, transfer) of Wansoft and Odoo, with full history, a normalized type, cost and a common product code (5.4).
- **Wansoft code of an Odoo product:** new table `analytics_odoo_product_code`, refreshed every night (5.4). Corrects version 1.3, which pointed at `inventory_mapping_dictionary`, a table that is not granted.
- **Section 6 corrected:** `odoo_inventory_quant_daily` is to be used; the list of tables not to use no longer includes it.
- **Access:** the 4 tables that were missing are now granted (`costs_source_by_company`, `odoo_inventory_quant_daily` and the two `wansoft_parallel_*`) plus the 2 new ones; 28 `wansoft` tables in total, all in Appendix A.

- **Section 9 updated:** the unified movements are live; next steps: access limited to a fixed IP and an encrypted connection (TLS).

Version 1.3 (2026-10-13)

- **Odoo stock day by day:** new table `odoo_inventory_quant_daily`, from 2026-10-08 (Sections 4 and 5.4).
- **Wansoft stock on a past date:** rule and verified example; exits without `Fecha` are dated by `FechaReal` (5.4).
- **Parallel Wansoft inventory** of the 8 branches migrated to Odoo: new tables `wansoft_parallel_inputinventory_entrada` and `wansoft_parallel_outgoinginventory_salida`, for comparison only (5.4).
- **Cash closings that arrive one day late** in 7 branches (Section 7).
- **New Section 9:** what is under construction (unified inventory movements, daily stock, supplier invoices) and Aeropuerto and Taquería Viaducto moving to Odoo on 2026-11-01.
- **New Appendix A:** every column (type, nulls, keys) of the 30 tables the user can read.
- **Access:** 3 more tables granted (`odoo_inventory_quant_daily` and the two `wansoft_parallel_*`); 26 `wansoft` tables in total.

Version 1.2 (2026-10-07)

- Costs: each branch's cost source as data (new table `costs_source_by_company`), the automatic per-branch switch from Wansoft to Odoo, and how to tell whether recent costs are complete (5.2).
- Purchases: unclassified purchases count in the business views, with the new column `catalog_status`; when a purchase appears (Odoo when confirmed, Wansoft when the invoice arrives) (5.3).
- Freshness and schedule with the facts of the first production week (Section 7).

Version 1.1 (2026-10-01)

- Production start on 2026-10-01: the migrated branches take purchases and inventory from Odoo from that date (Section 3).
- Costs: each branch on Odoo takes its cost from Odoo from its start date; cost history of Puebla and CentroMyJ (5.2).
- Warning: Odoo and Wansoft purchases do not match each other for the same branch and month (5.3).

Version 1.0 (2026-09-28)

- First edition, Spanish and English with PDF: table per question, golden rules, branch crosswalk, available history, domains, tables not to use, freshness and access.

Appendix A. Table schemas

Columns of every table the read-only user can query, taken from the production schema on 2026-10-16. **Null**: whether the column accepts NULL. **Key**: `PRI` primary key, `UNI` part of a unique key, `MUL` part of an index. The meaning of the important fields is in the section shown in brackets.

A.1 Sales

`getallordenesbyday_new_venta`: ticket header (5.1). 24 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`Sucursal`	varchar(255)	yes	—
`Venta_Id`	varchar(255)	yes	—
`Movimento`	varchar(255)	yes	MUL
`Orden`	varchar(255)	yes	—
`Mesa`	varchar(255)	yes	—
`Fecha`	varchar(255)	yes	—
`Mesero`	varchar(255)	yes	—
`Subtotal`	varchar(255)	yes	—
`IVA`	varchar(255)	yes	—
`IEPS`	varchar(255)	yes	—
`Total`	varchar(255)	yes	—
`Terminal`	varchar(255)	yes	—
`Personas`	varchar(255)	yes	—
`Descuento`	varchar(255)	yes	—
`Impuesto`	varchar(255)	yes	—
`MontoDescontado`	varchar(255)	yes	—
`TipoOrden`	varchar(255)	yes	—
`HoraApertura`	varchar(255)	yes	—
`HoraCierre`	varchar(255)	yes	—
`Moneda`	varchar(255)	yes	—
`CodigoCliente`	varchar(255)	yes	—
`Estatus`	varchar(255)	yes	—
`Ventas_Id`	varchar(255)	yes	—

`getallordenesbyday_new_detalleventa`: ticket lines (5.1). 30 columns.

Column	Type	Null	Key
id	int(11)	no	PRI
Sucursal	varchar(255)	yes	—
Movimiento_Id	varchar(255)	yes	MUL
DetalleVenta_Id	varchar(255)	yes	—
Descripcion	varchar(255)	yes	—
Cantidad	varchar(255)	yes	—
PrecioUnitario	varchar(255)	yes	—
Descuento	varchar(255)	yes	—
Subtotal	varchar(255)	yes	—
IVA	varchar(255)	yes	—
IEPS	varchar(255)	yes	—
Total	varchar(255)	yes	—
Modificador	varchar(255)	yes	—
Hora	varchar(255)	yes	—
Costo	varchar(255)	yes	—
TipoGrupo	varchar(255)	yes	—
CodigoTipoGrupo	varchar(255)	yes	—
CuentaContableTipoGrupo	varchar(255)	yes	—
Grupo	varchar(255)	yes	—
CodigoGrupo	varchar(255)	yes	—
CuentaContableGrupo	varchar(255)	yes	—
CodigoPlatillo	varchar(255)	yes	—
CuentaContablePlatillo	varchar(255)	yes	—
Platillo	varchar(255)	yes	—
ConIVA	varchar(255)	yes	—
ComandaId	varchar(255)	yes	—
TipoPlatilloId	varchar(255)	yes	—
TipoPromocionId	varchar(255)	yes	—
Cortesia	varchar(255)	yes	—
DetallesVenta_Id	varchar(255)	yes	—

`getallordenesbyday_new_pago`: ticket payments (5.1). 17 columns.

Column	Type	Null	Key

id	int(11)	no	PRI
Sucursal	varchar(255)	yes	—
Movimiento_Id	varchar(255)	yes	—
Fecha	varchar(255)	yes	—
IdMetodoDePago	varchar(255)	yes	—
MetodoDePago	varchar(255)	yes	—
CodigoMetodoDePago	varchar(255)	yes	—
ClaveSATMetodoDePago	varchar(255)	yes	—
CuentaContableMetodoDePago	varchar(255)	yes	—
Terminal	varchar(255)	yes	—
Total	varchar(255)	yes	—
Propina	varchar(255)	yes	—
Equivalencia	varchar(255)	yes	—
Moneda	varchar(255)	yes	—
MontoRecibidoEnMoneda	varchar(255)	yes	—
PagoAnticipadoId	varchar(255)	yes	—
Pagos_Id	varchar(255)	yes	MUL

`getallordenesbyday_new_modificador`: line modifiers (5.1). 28 columns.

Column	Type	Null	Key
id	int(11)	no	PRI
Sucursal	varchar(255)	yes	—
Movimiento_Id	varchar(255)	yes	MUL
Descripcion	varchar(255)	yes	—
Cantidad	varchar(255)	yes	—
PrecioUnitario	varchar(255)	yes	—
Descuento	varchar(255)	yes	—
Subtotal	varchar(255)	yes	—
IVA	varchar(255)	yes	—
IEPS	varchar(255)	yes	—
Total	varchar(255)	yes	—
Modificador	varchar(255)	yes	—
Hora	varchar(255)	yes	—
Costo	varchar(255)	yes	—

`TipoGrupo`	varchar(255)	yes	—
`CodigoTipoGrupo`	varchar(255)	yes	—
`CuentaContableTipoGrupo`	varchar(255)	yes	—
`Grupo`	varchar(255)	yes	—
`CodigoGrupo`	varchar(255)	yes	—
`CuentaContableGrupo`	varchar(255)	yes	—
`CodigoPlatillo`	varchar(255)	yes	—
`CuentaContablePlatillo`	varchar(255)	yes	—
`Platillo`	varchar(255)	yes	—
`ConIVA`	varchar(255)	yes	—
`ComandaId`	varchar(255)	yes	—
`TipoPlatilloId`	varchar(255)	yes	—
`TipoPromocionId`	varchar(255)	yes	—
`Modificadores_Id`	varchar(255)	yes	—

`getglobalcashclosing`: cash closing (Z report) per branch and day (5.1). 49 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_id`	int(11)	yes	MUL
`subsidiary_name`	varchar(255)	yes	—
`fecha_corte`	datetime	yes	—
`usuario`	varchar(255)	yes	—
`subtotal`	decimal(10,2)	yes	—
`iva`	decimal(10,2)	yes	—
`ieps`	decimal(10,2)	yes	—
`total_ventas`	decimal(10,2)	yes	—
`efectivo_por_ventas`	decimal(10,2)	yes	—
`efectivo_por_propina`	decimal(10,2)	yes	—
`fondo_de_caja`	decimal(10,2)	yes	—
`efectivo_real`	decimal(10,2)	yes	—
`no_ordenes`	int(11)	yes	—
`no_platillos`	int(11)	yes	—
`total_personas`	int(11)	yes	—
`promedio_platillos_orden`	decimal(10,2)	yes	—

promedio_por_orden	decimal(10,2)	yes	—
promedio_por_persona	decimal(10,2)	yes	—
total_ordenes_para_llevar	int(11)	yes	—
total_mesas_atendidas	int(11)	yes	—
total_ordenes_a_domicilio	int(11)	yes	—
total_ordenes_recoger	int(11)	yes	—
no_cortesias_en_cuentas	int(11)	yes	—
cortesias_en_cuentas	decimal(10,2)	yes	—
no_cortesias_en_platillos	int(11)	yes	—
cortesias_en_platillos	decimal(10,2)	yes	—
no_cancelaciones_en_cuentas	int(11)	yes	—
cancelaciones_en_cuentas	decimal(10,2)	yes	—
no_cancelaciones_en_platillos	int(11)	yes	—
cancelaciones_en_platillos	decimal(10,2)	yes	—
no_descuentos_en_cuentas	int(11)	yes	—
descuentos_en_cuentas	decimal(10,2)	yes	—
no_descuentos_en_platillos	int(11)	yes	—
descuentos_en_platillos	decimal(10,2)	yes	—
no_anulaciones_en_cuentas	int(11)	yes	—
anulaciones_en_cuentas	decimal(10,2)	yes	—
no_anulaciones_en_platillos	int(11)	yes	—
anulaciones_en_platillos	decimal(10,2)	yes	—
no_dxu_platillos	int(11)	yes	—
dxu_platillos	decimal(10,2)	yes	—
no_descuentos_megapuntos	int(11)	yes	—
descuentos_megapuntos	decimal(10,2)	yes	—
no_promociones	int(11)	yes	—
promociones	decimal(10,2)	yes	—
no_cupones	int(11)	yes	—
cupones	decimal(10,2)	yes	—
mes_ano	varchar(7)	yes	—
created_at	timestamp	no	—

A.2 Costs

`costeomensual`: month-to-date cost, daily snapshot (5.2). 17 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_id`	int(11)	yes	MUL
`subsidiary_name`	varchar(255)	yes	—
`CostoTotal`	decimal(10,2)	yes	—
`CostoDeProductosVendidos`	decimal(10,2)	yes	—
`CostoIdealDeProductosPendientesDeRebaja`	decimal(10,2)	yes	—
`CostoDeCortesías`	decimal(10,2)	yes	—
`CostoDeCancelaciones`	decimal(10,2)	yes	—
`CostoDeMerma`	decimal(10,2)	yes	—
`CostoDeDesperdicio`	decimal(10,2)	yes	—
`CostoDeRobo`	decimal(10,2)	yes	—
`CostoDeConsumo`	decimal(10,2)	yes	—
`AjustePorSobrantes`	decimal(10,2)	yes	—
`UtilidadMarginal`	decimal(10,2)	yes	—
`mes_ano`	varchar(7)	yes	—
`created_at`	timestamp	no	—
`created_date`	date	yes	—

`costeomensual_semanapyq`: week-to-date cost (5.2). 17 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_id`	int(11)	yes	MUL
`subsidiary_name`	varchar(255)	yes	—
`CostoTotal`	decimal(10,2)	yes	—
`CostoDeProductosVendidos`	decimal(10,2)	yes	—
`CostoIdealDeProductosPendientesDeRebaja`	decimal(10,2)	yes	—
`CostoDeCortesías`	decimal(10,2)	yes	—
`CostoDeCancelaciones`	decimal(10,2)	yes	—
`CostoDeMerma`	decimal(10,2)	yes	—
`CostoDeDesperdicio`	decimal(10,2)	yes	—
`CostoDeRobo`	decimal(10,2)	yes	—

`CostoDeConsumo`	decimal(10,2)	yes	—
`AjustePorSobrantes`	decimal(10,2)	yes	—
`UtilidadMarginal`	decimal(10,2)	yes	—
`mes_ano`	varchar(7)	yes	—
`created_at`	timestamp	no	—
`created_date`	date	yes	—

`gettotalcostbydate`: cost of sales of the day (5.2). 7 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_id`	int(11)	yes	MUL
`subsidiary_name`	varchar(255)	yes	—
`CostoTotalVenta`	decimal(10,2)	yes	—
`mes_ano`	varchar(7)	yes	—
`created_at`	timestamp	no	—
`created_date`	date	yes	—

`gettablajeriareport`: butchery yields, Wansoft history only (5.2). 17 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_id`	int(11)	yes	—
`subsidiary_name`	varchar(255)	yes	—
`InputDate`	date	yes	—
`UserName`	varchar(255)	yes	—
`QuantityOfBaseProduct`	decimal(15,10)	yes	—
`ProductBase`	varchar(255)	yes	—
`ProductBaseCost`	decimal(15,4)	yes	—
`UnitOfMeasureOfBaseProduct`	varchar(50)	yes	—
`QuantityDecrease`	decimal(15,10)	yes	—
`Warehouse`	varchar(255)	yes	—
`QuantityOfGeneratedProduct`	decimal(15,10)	yes	—
`GeneratedProduct`	varchar(255)	yes	—
`UnitCostOfGeneratedProduct`	decimal(15,10)	yes	—
`totalCostOfGeneratedProduct`	decimal(15,10)	yes	—

`RegistrationDate`	date	yes	—
`created_at`	timestamp	no	—

`costs_source_by_company`: cost source per branch (5.2). 6 columns.

Column	Type	Null	Key
`company_source_key`	varchar(100)	no	PRI
`wansoft_subsidiary_id`	int(11)	yes	—
`odoo_company_id`	int(11)	yes	—
`odoo_cost_start_date`	date	yes	—
`reason`	varchar(30)	no	—
`updated_at`	datetime	no	—

A.3 Purchases and invoices

`analytics_purchase_order_lines`: purchase lines, Wansoft + Odoo (5.3). 58 columns.

Column	Type	Null	Key
`purchase_order_line_analytical_key`	bigint(20)	no	PRI
`canonical_purchase_order_line_id`	bigint(20)	no	UNI
`source_system`	varchar(50)	no	MUL
`source_domain`	varchar(100)	yes	—
`source_order_line_id`	varchar(100)	yes	—
`source_order_id`	varchar(100)	yes	—
`purchase_order_name`	varchar(255)	yes	—
`company_source_key`	varchar(255)	yes	MUL
`company_analytical_key`	bigint(20)	yes	—
`company_id`	varchar(100)	yes	—
`company_name`	varchar(255)	yes	—
`final_purchase_source_status`	varchar(100)	yes	—
`company_migration_type`	varchar(100)	yes	—
`history_source`	varchar(100)	yes	—
`include_odoo_history`	tinyint(1)	yes	—
`operational_start_date`	date	yes	—
`migration_policy_source`	varchar(100)	yes	—
`order_date`	datetime	yes	—
`order_date_key`	int(11)	yes	—

vendor_analytical_key	bigint(20)	yes	MUL
vendor_id	varchar(100)	yes	—
vendor_name	varchar(255)	yes	—
normalized_vendor_name	varchar(255)	yes	—
is_internal_vendor	tinyint(1)	no	—
include_vendor_in_business_views	tinyint(1)	no	—
product_analytical_key	bigint(20)	yes	MUL
product_id	varchar(100)	yes	—
product_name	varchar(255)	yes	—
wansoft_code	varchar(100)	yes	—
wansoft_product_name	varchar(255)	yes	—
wansoft_department	varchar(255)	yes	—
product_mapping_found	tinyint(1)	yes	—
product_mapping_status	varchar(100)	yes	—
product_mapping_source	varchar(100)	yes	—
purchase_line_type	varchar(100)	yes	—
purchase_product_scope	varchar(100)	yes	—
purchase_mapping_bucket	varchar(100)	yes	—
purchase_classification_source	varchar(100)	yes	—
extracted_product_code	varchar(100)	yes	—
product_identity_status	varchar(100)	yes	—
dim_product_mapping_status	varchar(100)	yes	—
is_product_mapped	tinyint(1)	no	—
is_product_review_required	tinyint(1)	no	—
include_product_in_business_views	tinyint(1)	no	—
product_qty	decimal(18,4)	yes	—
qty_received	decimal(18,4)	yes	—
qty_invoiced	decimal(18,4)	yes	—
price_unit	decimal(18,4)	yes	—
price_subtotal	decimal(18,4)	yes	—
price_total	decimal(18,4)	yes	—
state	varchar(100)	yes	—
canonical_loaded_at	timestamp	yes	—

include_in_business_views	tinyint(1)	no	MUL
exclude_reason	varchar(500)	yes	—
line_review_status	varchar(100)	yes	—
catalog_status	varchar(50)	yes	—
created_at	datetime	no	—
updated_at	datetime	no	—

analytics_purchase_orders: purchase orders / invoices, Wansoft + Odoo (5.3). 43 columns.

Column	Type	Null	Key
purchase_order_analytical_key	bigint(20)	no	PRI
source_system	varchar(50)	no	MUL
source_domain	varchar(100)	yes	—
source_order_id	varchar(100)	no	—
purchase_order_name	varchar(255)	yes	—
company_source_key	varchar(255)	yes	MUL
company_analytical_key	bigint(20)	yes	—
company_id	varchar(100)	yes	—
company_name	varchar(255)	yes	—
final_purchase_source_status	varchar(100)	yes	—
company_migration_type	varchar(100)	yes	—
history_source	varchar(100)	yes	—
include_odoo_history	tinyint(1)	yes	—
operational_start_date	date	yes	—
migration_policy_source	varchar(100)	yes	—
order_date	datetime	yes	—
order_date_key	int(11)	yes	—
vendor_analytical_key	bigint(20)	yes	MUL
vendor_id	varchar(100)	yes	—
vendor_name	varchar(255)	yes	—
normalized_vendor_name	varchar(255)	yes	—
is_internal_vendor	tinyint(1)	no	—
include_vendor_in_business_views	tinyint(1)	no	—
line_count	bigint(20)	no	—
business_line_count	bigint(20)	no	—

excluded_line_count	bigint(20)	no	—
review_required_line_count	bigint(20)	no	—
internal_vendor_line_count	bigint(20)	no	—
review_required_product_line_count	bigint(20)	no	—
orphan_product_line_count	bigint(20)	no	—
product_qty_total	decimal(18,4)	yes	—
qty_received_total	decimal(18,4)	yes	—
qty_invoiced_total	decimal(18,4)	yes	—
price_subtotal_total	decimal(18,4)	yes	—
price_total_total	decimal(18,4)	yes	—
state_values	text	yes	—
canonical_loaded_at_min	timestamp	yes	—
canonical_loaded_at_max	timestamp	yes	—
include_in_business_views	tinyint(1)	no	MUL
exclude_reason	varchar(500)	yes	—
order_review_status	varchar(100)	yes	—
created_at	datetime	no	—
updated_at	datetime	no	—

analytics_purchase_daily_company_product: purchases per branch × day × product (5.3). 42 columns.

Column	Type	Null	Key
daily_company_product_key	bigint(20)	no	PRI
company_source_key	varchar(255)	no	MUL
company_analytical_key	bigint(20)	yes	—
order_date	date	yes	—
order_date_key	int(11)	no	—
product_analytical_key	bigint(20)	yes	MUL
product_analytical_group_key	bigint(20)	no	MUL
source_system	varchar(50)	no	MUL
product_identity_status	varchar(100)	yes	—
dim_product_mapping_status	varchar(100)	yes	—
is_product_mapped	tinyint(1)	no	—
is_product_review_required	tinyint(1)	no	—
include_product_in_business_views	tinyint(1)	no	—

`line_count`	bigint(20)	no	—
`business_line_count`	bigint(20)	no	—
`excluded_line_count`	bigint(20)	no	—
`review_required_line_count`	bigint(20)	no	—
`internal_vendor_line_count`	bigint(20)	no	—
`review_required_product_line_count`	bigint(20)	no	—
`orphan_product_line_count`	bigint(20)	no	—
`purchase_order_count`	bigint(20)	no	—
`vendor_count`	bigint(20)	no	—
`product_qty_total`	decimal(18,4)	yes	—
`qty_received_total`	decimal(18,4)	yes	—
`qty_invoiced_total`	decimal(18,4)	yes	—
`business_product_qty_total`	decimal(18,4)	yes	—
`business_qty_received_total`	decimal(18,4)	yes	—
`business_qty_invoiced_total`	decimal(18,4)	yes	—
`excluded_product_qty_total`	decimal(18,4)	yes	—
`excluded_qty_received_total`	decimal(18,4)	yes	—
`excluded_qty_invoiced_total`	decimal(18,4)	yes	—
`price_subtotal_total`	decimal(18,4)	yes	—
`price_total_total`	decimal(18,4)	yes	—
`business_price_subtotal_total`	decimal(18,4)	yes	—
`business_price_total_total`	decimal(18,4)	yes	—
`excluded_price_subtotal_total`	decimal(18,4)	yes	—
`excluded_price_total_total`	decimal(18,4)	yes	—
`include_in_business_views`	tinyint(1)	no	MUL
`exclude_reason`	varchar(500)	yes	—
`aggregate_review_status`	varchar(100)	yes	MUL
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

`getexpenses_factura`: Wansoft supplier invoices by account (5.3). 37 columns.

Column	Type	Null	Key
`id`	int(11)	no	—
`Sucursal`	varchar(255)	yes	—

`Factura_Id`	varchar(255)	yes	—
`IdDocumento`	varchar(255)	yes	—
`Folio`	varchar(255)	yes	—
`RFCProveedor`	varchar(255)	yes	—
`NombreProveedor`	varchar(255)	yes	—
`ClaveProveedor`	varchar(255)	yes	—
`CuentaContableProveedor`	varchar(255)	yes	—
`Subtotal`	varchar(255)	yes	—
`IVA`	varchar(255)	yes	—
`IEPS`	varchar(255)	yes	—
`Total`	varchar(255)	yes	—
`FechaDeExpedicion`	varchar(255)	yes	—
`FechaDeExpiracion`	varchar(255)	yes	—
`TerminosDePago`	varchar(255)	yes	—
`Cuenta`	varchar(255)	yes	—
`Subcuenta`	varchar(255)	yes	—
`Estatus`	varchar(255)	yes	—
`TotalDeudor`	varchar(255)	yes	—
`TipoDeEgreso`	varchar(255)	yes	—
`UUID`	varchar(255)	yes	—
`IdOrdenCompra`	varchar(255)	yes	—
`FolioOrdenCompra`	varchar(255)	yes	—
`FechaDeRegistro`	varchar(255)	yes	—
`DiasCredito`	varchar(255)	yes	—
`ColoniaProveedor`	varchar(255)	yes	—
`CiudadProveedor`	varchar(255)	yes	—
`CPProveedor`	varchar(255)	yes	—
`TelefonoProveedor`	varchar(255)	yes	—
`CorreoProveedor`	varchar(255)	yes	—
`CalleProveedor`	varchar(255)	yes	—
`NumeroIntProveedor`	varchar(255)	yes	—
`NumeroExtProveedor`	varchar(255)	yes	—
`Discount`	varchar(255)	yes	—

`Retentions`	varchar(255)	yes	—
`Egresos_Id`	varchar(255)	yes	—

A.4 Inventory

`analytics_inventory_balance`: today's stock, all 19 branches (5.4). 15 columns.

Column	Type	Null	Key
`inventory_balance_key`	bigint(20)	no	PRI
`company_source_key`	varchar(255)	no	MUL
`source_system`	varchar(20)	no	—
`wansoft_code`	varchar(255)	no	MUL
`product_name`	varchar(500)	yes	—
`current_balance_qty`	decimal(18,4)	no	—
`source_row_count`	bigint(20)	no	—
`entrada_qty`	decimal(18,4)	yes	—
`salida_qty`	decimal(18,4)	yes	—
`final_inventory_source_status`	varchar(100)	no	—
`include_in_business_views`	tinyint(1)	no	MUL
`exclude_reason`	varchar(500)	yes	—
`refreshed_at`	datetime	no	—
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

`analytics_inventory_current_product_location`: today's Odoo stock by location (5.4). 37 columns.

Column	Type	Null	Key
`inventory_current_product_location_key`	bigint(20)	no	PRI
`current_snapshot_loaded_at`	datetime	no	MUL
`snapshot_date`	date	yes	—
`snapshot_date_key`	int(11)	no	MUL
`product_analytical_key`	bigint(20)	no	MUL
`odoo_product_id`	varchar(100)	yes	—
`odoo_product_name`	varchar(500)	yes	—
`product_code`	varchar(255)	yes	—
`wansoft_code`	varchar(255)	yes	—
`wansoft_product_name`	varchar(500)	yes	—

wansoft_department	varchar(255)	yes	—
product_identity_status	varchar(100)	yes	—
dim_product_mapping_status	varchar(100)	yes	—
is_product_mapped	tinyint(1)	no	—
is_product_review_required	tinyint(1)	no	—
include_product_in_business_views	tinyint(1)	no	—
source_location_id	varchar(100)	no	MUL
location_name	varchar(500)	yes	—
normalized_location_name	varchar(500)	yes	—
location_usage_type	varchar(100)	yes	—
is_virtual_location	tinyint(1)	no	—
is_partner_location	tinyint(1)	no	—
is_internal_location	tinyint(1)	no	—
location_mapping_status	varchar(100)	yes	—
company_source_key	varchar(255)	yes	—
company_mapping_status	varchar(100)	yes	—
source_row_count	bigint(20)	no	—
current_stock_qty	decimal(18,4)	no	—
positive_stock_qty	decimal(18,4)	no	—
zero_stock_row_count	bigint(20)	no	—
negative_stock_qty	decimal(18,4)	no	—
negative_row_count	bigint(20)	no	—
include_in_business_views	tinyint(1)	no	MUL
exclude_reason	varchar(500)	yes	—
aggregate_review_status	varchar(100)	no	MUL
created_at	datetime	no	—
updated_at	datetime	no	—

getinputinventory_entrada: Wansoft inventory entries (5.4). 38 columns.

Column	Type	Null	Key
id	int(11)	no	PRI
subsidiary_name	varchar(255)	yes	—
IdEntrada	varchar(50)	yes	MUL
ClaveAlmacen	varchar(50)	yes	—

`Almacen`	varchar(255)	yes	—
`IdAlmacen`	int(11)	yes	—
`CuentaContableAlmacen`	varchar(255)	yes	—
`CodigoDepartamento`	varchar(255)	yes	—
`Departamento`	varchar(255)	yes	—
`CuentaContableDepartamento`	varchar(255)	yes	—
`IdProducto`	int(11)	yes	—
`CodigoProducto`	varchar(50)	yes	—
`NombreProducto`	varchar(255)	yes	—
`CodigoUnidadDeMedida`	varchar(50)	yes	—
`IdUnidadDeMedida`	int(11)	yes	—
`UnidadDeMedida`	varchar(50)	yes	—
`TipoEntrada`	varchar(50)	yes	—
`Cantidad`	decimal(15,10)	yes	—
`CostoUnitario`	decimal(15,4)	yes	—
`ProductoConIVA`	tinyint(1)	yes	—
`Caducidad`	date	yes	—
`FechaEntrada`	datetime	yes	MUL
`Factura`	varchar(50)	yes	—
`FechaFactura`	date	yes	—
`RFCProveedor`	varchar(50)	yes	—
`ClaveProveedor`	varchar(50)	yes	—
`NombreProveedor`	varchar(255)	yes	—
`IdTransferencia`	int(11)	yes	—
`FolioTransferencia`	varchar(50)	yes	—
`IdOrdenCompra`	int(11)	yes	—
`FolioOrdenCompra`	varchar(50)	yes	—
`RFCProveedorOrdenCompra`	varchar(50)	yes	—
`ProveedorOrdenCompra`	varchar(255)	yes	—
`IdDocumento`	int(11)	yes	—
`IdUsuario`	int(11)	yes	—
`NombreUsuario`	varchar(255)	yes	—
`FechaReal`	date	yes	—

`created_at`	timestamp	no	—

`getoutgoinginventory_salida`: Wansoft inventory exits (5.4). 29 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_name`	varchar(255)	yes	MUL
`IdSalida`	varchar(50)	yes	—
`IdEntrada`	varchar(50)	yes	—
`IdAlmacen`	int(11)	yes	—
`Almacen`	varchar(255)	yes	—
`CuentaContableAlmacen`	varchar(255)	yes	—
`CuentaContableDepartamento`	varchar(255)	yes	—
`Departamento`	varchar(255)	yes	—
`IdProducto`	int(11)	yes	—
`CodigoProducto`	varchar(50)	yes	—
`NombreProducto`	varchar(255)	yes	—
`CodigoUnidadDeMedida`	varchar(50)	yes	—
`IdUnidadDeMedida`	int(11)	yes	—
`UnidadDeMedida`	varchar(50)	yes	—
`TipoSalida`	varchar(50)	yes	—
`Cantidad`	decimal(15,10)	yes	—
`CostoUnitario`	decimal(15,4)	yes	—
`Caducidad`	date	yes	—
`FechaSalida`	datetime	yes	MUL
`IdTransferencia`	varchar(50)	yes	—
`FolioTransferencia`	varchar(50)	yes	—
`Orden`	varchar(50)	yes	—
`Fecha`	date	yes	—
`IdDetalleVenta`	varchar(50)	yes	—
`IdUsuario`	varchar(50)	yes	—
`NombreUsuario`	varchar(255)	yes	—
`FechaReal`	date	yes	—
`created_at`	timestamp	no	—

`wansoft_parallel_inputinventory_entrada`: entries the Odoo branches keep capturing in Wansoft, comparison only (5.4). 38 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_name`	varchar(255)	yes	—
`IdEntrada`	varchar(50)	yes	MUL
`ClaveAlmacen`	varchar(50)	yes	—
`Almacen`	varchar(255)	yes	—
`IdAlmacen`	int(11)	yes	—
`CuentaContableAlmacen`	varchar(255)	yes	—
`CodigoDepartamento`	varchar(255)	yes	—
`Departamento`	varchar(255)	yes	—
`CuentaContableDepartamento`	varchar(255)	yes	—
`IdProducto`	int(11)	yes	—
`CodigoProducto`	varchar(50)	yes	—
`NombreProducto`	varchar(255)	yes	—
`CodigoUnidadDeMedida`	varchar(50)	yes	—
`IdUnidadDeMedida`	int(11)	yes	—
`UnidadDeMedida`	varchar(50)	yes	—
`TipoEntrada`	varchar(50)	yes	—
`Cantidad`	decimal(15,10)	yes	—
`CostoUnitario`	decimal(15,4)	yes	—
`ProductoConIVA`	tinyint(1)	yes	—
`Caducidad`	date	yes	—
`FechaEntrada`	datetime	yes	MUL
`Factura`	varchar(50)	yes	—
`FechaFactura`	date	yes	—
`RFCProveedor`	varchar(50)	yes	—
`ClaveProveedor`	varchar(50)	yes	—
`NombreProveedor`	varchar(255)	yes	—
`IdTransferencia`	int(11)	yes	—
`FolioTransferencia`	varchar(50)	yes	—
`IdOrdenCompra`	int(11)	yes	—

`FolioOrdenCompra`	varchar(50)	yes	—
`RFCProveedorOrdenCompra`	varchar(50)	yes	—
`ProveedorOrdenCompra`	varchar(255)	yes	—
`IdDocumento`	int(11)	yes	—
`IdUsuario`	int(11)	yes	—
`NombreUsuario`	varchar(255)	yes	—
`FechaReal`	date	yes	—
`created_at`	timestamp	no	—

`wansoft_parallel_outgoinginventory_salida`: exits the Odoo branches keep capturing in Wansoft, comparison only (5.4). 29 columns.

Column	Type	Null	Key
`id`	int(11)	no	PRI
`subsidiary_name`	varchar(255)	yes	MUL
`IdSalida`	varchar(50)	yes	—
`IdEntrada`	varchar(50)	yes	—
`IdAlmacen`	int(11)	yes	—
`Almacen`	varchar(255)	yes	—
`CuentaContableAlmacen`	varchar(255)	yes	—
`CuentaContableDepartamento`	varchar(255)	yes	—
`Departamento`	varchar(255)	yes	—
`IdProducto`	int(11)	yes	—
`CodigoProducto`	varchar(50)	yes	—
`NombreProducto`	varchar(255)	yes	—
`CodigoUnidadDeMedida`	varchar(50)	yes	—
`IdUnidadDeMedida`	int(11)	yes	—
`UnidadDeMedida`	varchar(50)	yes	—
`TipoSalida`	varchar(50)	yes	—
`Cantidad`	decimal(15,10)	yes	—
`CostoUnitario`	decimal(15,4)	yes	—
`Caducidad`	date	yes	—
`FechaSalida`	datetime	yes	MUL
`IdTransferencia`	varchar(50)	yes	—
`FolioTransferencia`	varchar(50)	yes	—

`Orden`	varchar(50)	yes	—
`Fecha`	date	yes	—
`IdDetalleVenta`	varchar(50)	yes	—
`IdUsuario`	varchar(50)	yes	—
`NombreUsuario`	varchar(255)	yes	—
`FechaReal`	date	yes	—
`created_at`	timestamp	no	—

`odoo_inventory_quant_daily`: Odoo stock day by day, since 2026-10-08 (5.4). 14 columns.

Column	Type	Null	Key
`stock_date`	date	no	PRI
`odoo_location_id`	int(11)	no	PRI
`odoo_product_id`	int(11)	no	PRI
`captured_at`	datetime	no	—
`odoo_company_id`	int(11)	yes	—
`odoo_company_name`	varchar(255)	yes	—
`company_source_key`	varchar(100)	yes	MUL
`location_name`	varchar(500)	yes	—
`location_usage`	varchar(30)	no	—
`product_code`	varchar(100)	yes	—
`product_name`	varchar(500)	yes	—
`quantity`	decimal(18,4)	no	—
`reserved_quantity`	decimal(18,4)	no	—
`quant_count`	int(11)	no	—

`analytics_inventory_movements`: unified inventory movements, Wansoft + Odoo (5.4). 24 columns.

Column	Type	Null	Key
`movement_key`	bigint(20)	no	PRI
`movement_date`	date	no	—
`company_source_key`	varchar(100)	no	MUL
`source_system`	varchar(10)	no	MUL
`source_record`	varchar(20)	no	MUL
`source_movement_id`	varchar(80)	no	—
`captured_date`	date	yes	—

direction	char(3)	no	—
movement_type	varchar(20)	no	—
source_movement_type	varchar(100)	yes	—
wansoft_code	varchar(50)	yes	—
odoo_product_id	int(11)	yes	—
product_name	varchar(255)	yes	—
unit_name	varchar(50)	yes	—
quantity	decimal(18,6)	no	—
unit_cost	decimal(18,6)	yes	—
total_cost	decimal(18,4)	yes	—
location_name	varchar(255)	yes	—
counts_in_stock	tinyint(1)	no	—
final_source_status	varchar(40)	no	—
include_in_business_views	tinyint(1)	no	—
loaded_at	datetime	no	—
product_code	varchar(60)	yes	MUL
product_code_status	varchar(15)	yes	—

analytics_odoo_product_code: Wansoft code of each Odoo product (5.4). 6 columns.

Column	Type	Null	Key
odoo_product_id	int(11)	no	PRI
odoo_product_name	varchar(255)	yes	—
wansoft_code	varchar(50)	no	MUL
wansoft_product_name	varchar(255)	yes	—
mapping_source	varchar(50)	yes	—
updated_at	datetime	no	—

A.5 Governance and dimensions

analytics_company_domain_coverage: source of each domain per branch (5.6). 31 columns.

Column	Type	Null	Key
company_source_key	varchar(255)	no	PRI
display_name	varchar(255)	no	—
is_active_branch	tinyint(1)	no	—
is_internal_provider	tinyint(1)	no	—

`is_final_operating_branch`	tinyint(1)	no	—
`is_future_rollout`	tinyint(1)	no	—
`is_zenput_location`	tinyint(1)	no	—
`is_zenput_only`	tinyint(1)	no	—
`purchases_source_system`	varchar(100)	no	—
`inventory_source_system`	varchar(100)	no	—
`sales_source_system`	varchar(100)	no	—
`zenput_source_status`	varchar(100)	no	—
`rollout_type`	varchar(100)	yes	MUL
`rollout_status`	varchar(100)	yes	—
`operational_start_date`	date	yes	—
`has_purchases`	tinyint(1)	no	MUL
`has_purchase_orders`	tinyint(1)	no	—
`has_purchase_lines`	tinyint(1)	no	—
`has_inventory`	tinyint(1)	no	—
`has_zenput_submissions`	tinyint(1)	no	—
`has_zenput_tasks`	tinyint(1)	no	—
`has_sales_future_placeholder`	tinyint(1)	no	—
`purchase_order_count`	bigint(20)	no	—
`purchase_line_count`	bigint(20)	no	—
`inventory_snapshot_count`	bigint(20)	no	—
`zenput_submission_count`	bigint(20)	no	—
`zenput_task_count`	bigint(20)	no	—
`coverage_status`	varchar(100)	no	MUL
`coverage_notes`	text	yes	—
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

`dim_company_analytical`: branches and companies (5.6). 25 columns.

Column	Type	Null	Key
`company_analytical_key`	bigint(20)	no	PRI
`company_source_key`	varchar(255)	no	UNI
`display_name`	varchar(255)	no	MUL
`normalized_name`	varchar(255)	yes	—

`brand_group`	varchar(255)	yes	—
`is_active_branch`	tinyint(1)	no	—
`is_internal_provider`	tinyint(1)	no	—
`is_final_operating_branch`	tinyint(1)	no	—
`is_future_rollout`	tinyint(1)	no	—
`is_wansoft_company`	tinyint(1)	no	—
`is_odoo_company`	tinyint(1)	no	—
`is_zenput_location`	tinyint(1)	no	—
`is_zenput_only`	tinyint(1)	no	—
`purchases_source_system`	varchar(100)	no	MUL
`inventory_source_system`	varchar(100)	no	—
`sales_source_system`	varchar(100)	no	—
`zenput_source_status`	varchar(100)	no	—
`rollout_type`	varchar(100)	yes	MUL
`rollout_status`	varchar(100)	yes	—
`operational_start_date`	date	yes	—
`include_in_business_views`	tinyint(1)	no	—
`exclude_reason`	varchar(255)	yes	—
`notes`	text	yes	—
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

`dim_product`: products (5.3). 38 columns.

Column	Type	Null	Key
`product_analytical_key`	bigint(20)	no	PRI
`product_display_name`	varchar(500)	no	—
`normalized_product_name`	varchar(500)	no	MUL
`product_canonical_name`	varchar(500)	yes	—
`product_identity_status`	varchar(100)	no	MUL
`wansoft_code`	varchar(255)	yes	MUL
`wansoft_product_name`	varchar(500)	yes	—
`wansoft_department`	varchar(255)	yes	—
`wansoft_family`	varchar(255)	yes	—
`wansoft_group`	varchar(255)	yes	—

odoo_product_id	varchar(255)	yes	MUL
odoo_product_name	varchar(500)	yes	—
odoo_default_code	varchar(255)	yes	—
odoo_category	varchar(255)	yes	—
odoo_uom	varchar(100)	yes	—
mapping_status	varchar(100)	no	MUL
mapping_source	varchar(255)	yes	—
mapping_confidence	varchar(100)	yes	—
is_mapped	tinyint(1)	no	—
is_unmapped	tinyint(1)	no	—
is_review_required	tinyint(1)	no	—
is_excluded	tinyint(1)	no	—
exclude_reason	varchar(255)	yes	—
company_scope	varchar(255)	yes	—
scope_bucket	varchar(255)	yes	MUL
product_business_domain	varchar(255)	yes	—
is_restaurant_product	tinyint(1)	no	—
is_bodegon_product	tinyint(1)	no	—
is_empanadas_product	tinyint(1)	no	—
is_shared_cross_company	tinyint(1)	no	—
source_system	varchar(100)	no	MUL
source_table	varchar(255)	yes	—
source_product_key	varchar(255)	no	—
source_product_name	varchar(500)	yes	—
include_in_business_views	tinyint(1)	no	MUL
notes	text	yes	—
created_at	datetime	no	—
updated_at	datetime	no	—

dim_vendor: vendors (5.3). 18 columns.

Column	Type	Null	Key
vendor_analytical_key	bigint(20)	no	PRI
vendor_display_name	varchar(255)	no	MUL
normalized_vendor_name	varchar(255)	no	UNI

`vendor_canonical_name`	varchar(255)	yes	—
`vendor_source_system`	varchar(100)	no	MUL
`wansoft_vendor_id`	varchar(255)	yes	—
`odoo_vendor_id`	varchar(255)	yes	—
`source_vendor_name`	text	yes	—
`source_vendor_key`	text	yes	—
`is_internal_vendor`	tinyint(1)	no	MUL
`is_external_vendor`	tinyint(1)	no	—
`is_active`	tinyint(1)	no	—
`is_review_required`	tinyint(1)	no	—
`include_in_business_views`	tinyint(1)	no	MUL
`exclude_reason`	varchar(255)	yes	—
`notes`	text	yes	—
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

`dim_time`: calendar (5.6). 37 columns.

Column	Type	Null	Key
`date_key`	int(11)	no	PRI
`calendar_date`	date	no	UNI
`year`	smallint(6)	no	MUL
`year_start_date`	date	no	—
`year_end_date`	date	no	—
`is_year_start`	tinyint(1)	no	—
`is_year_end`	tinyint(1)	no	—
`quarter_number`	tinyint(4)	no	—
`quarter_name`	varchar(10)	no	—
`year_quarter`	varchar(20)	no	MUL
`quarter_start_date`	date	no	—
`quarter_end_date`	date	no	—
`is_quarter_start`	tinyint(1)	no	—
`is_quarter_end`	tinyint(1)	no	—
`month_number`	tinyint(4)	no	—
`month_name`	varchar(20)	no	—

`month_short_name`	varchar(10)	no	—
`year_month_label`	varchar(10)	no	MUL
`month_start_date`	date	no	—
`month_end_date`	date	no	—
`is_month_start`	tinyint(1)	no	—
`is_month_end`	tinyint(1)	no	—
`week_of_year`	tinyint(4)	no	—
`iso_week_of_year`	tinyint(4)	no	—
`iso_year`	smallint(6)	no	MUL
`week_start_date`	date	no	—
`week_end_date`	date	no	—
`is_week_start`	tinyint(1)	no	—
`is_week_end`	tinyint(1)	no	—
`day_of_month`	tinyint(4)	no	—
`day_of_year`	smallint(6)	no	—
`day_of_week_number`	tinyint(4)	no	—
`day_of_week_name`	varchar(20)	no	—
`day_of_week_short_name`	varchar(10)	no	—
`is_weekend`	tinyint(1)	no	MUL
`created_at`	datetime	no	—
`updated_at`	datetime	no	—

A.6 Zenput (database zenput)

`form_templates`: checklist templates (5.5). 8 columns.

Column	Type	Null	Key
`form_id`	int(11)	no	PRI
`title`	varchar(255)	yes	—
`num_submissions`	int(11)	yes	—
`date_created`	datetime	yes	—
`date_last_submitted`	datetime	yes	—
`creator_full_name`	varchar(255)	yes	—
`category_name`	varchar(255)	yes	—
`last_updated`	timestamp	no	—

`submissions`: submitted checklists (5.5). 6 columns.

Column	Type	Null	Key
`submission_id`	varchar(255)	no	PRI
`form_template_id`	int(11)	yes	MUL
`location_name`	varchar(255)	yes	—
`user_display_name`	varchar(255)	yes	—
`date_submitted`	datetime	yes	—
`time_to_complete`	int(11)	yes	—

`submission_answers`: answers (5.5). 6 columns.

Column	Type	Null	Key
`answer_id`	int(11)	no	PRI
`submission_id`	varchar(255)	yes	MUL
`field_id`	int(11)	yes	—
`title`	text	yes	—
`field_type`	varchar(50)	yes	—
`value_as_string`	text	yes	—

`zenput_tasks`: tasks (5.5). 42 columns.

Column	Type	Null	Key
`task_id`	int(11)	no	PRI
`title`	text	yes	—
`description`	text	yes	—
`company_id`	int(11)	yes	—
`account_id`	int(11)	yes	—
`account_name`	varchar(255)	yes	—
`account_address`	text	yes	—
`account_city`	varchar(100)	yes	—
`account_state`	varchar(100)	yes	—
`account_zipcode`	varchar(20)	yes	—
`account_country`	varchar(10)	yes	—
`account_lat`	decimal(10,8)	yes	—
`account_lon`	decimal(11,8)	yes	—
`status_id`	int(11)	yes	—

`status_type`	varchar(50)	yes	—
`status_name`	varchar(50)	yes	—
`reply_type`	varchar(50)	yes	—
`reporter_id`	int(11)	yes	—
`reporter_display_name`	varchar(255)	yes	—
`assignee_id`	int(11)	yes	—
`assignee_display_name`	varchar(255)	yes	—
`date_created`	datetime	yes	—
`date_modified`	datetime	yes	—
`date_start`	datetime	yes	—
`date_due`	datetime	yes	—
`time_zone`	varchar(100)	yes	—
`is_active`	tinyint(1)	yes	—
`is_closed`	tinyint(1)	yes	—
`project_id`	int(11)	yes	—
`fulfillment_type`	varchar(50)	yes	—
`fulfillment_date_completed`	datetime	yes	—
`fulfillment_date_submitted`	datetime	yes	—
`fulfillment_user_id`	int(11)	yes	—
`fulfillment_user_display_name`	varchar(255)	yes	—
`fulfillment_fields`	longtext	yes	—
`date_submitted`	datetime	yes	—
`deleted`	tinyint(1)	yes	—
`num_comments`	int(11)	yes	—
`is_completed_late`	tinyint(1)	yes	—
`current_state`	varchar(50)	yes	—
`subscribers`	longtext	yes	—
`last_updated`	timestamp	no	—